

CAHIER DES CHARGES FONCTIONNEL ET TECHNIQUE

TASK-FLOW

Plateforme de Gestion de Projets Collaborative

Projet Fil Rouge
17 janvier 2026

Rédigé par
Debbarh Mohammed

Table des matières

1	Présentation du projet	2
1.1	Présentation du contexte	2
1.2	Nom du site	2
1.3	Cible adressée par le site	2
1.4	Cadre	2
1.5	Périmètre du projet	2
1.6	Description	2
2	Description graphique et ergonomique	2
2.1	Charte graphique	2
2.2	Inspiration	3
2.3	Logo	3
3	Objectifs du site	3
4	Besoins Fonctionnels	3
4.1	Arborescence du site	3
4.2	Exposition des projets (Le Board)	3
4.3	Inscription et Gestion des Rôles (RBAC)	4
4.4	Création d'une tâche	4
4.5	Attribution et Workflow	4
4.6	Commentaires	4
4.7	Sécurité	4
5	Choix technologiques	4
5.1	Front-End	4
5.2	Back-End	4
5.3	Compatibilité	5
6	Planification du projet	5
6.1	Sprint 1 : Conception	5
6.2	Sprint 2 : Maquettage (UI/UX)	5
6.3	Sprint 3 : Back-End	5
6.4	Sprint 4 : Front-End	5
7	Conception UML	6
7.1	Diagramme des Cas d'Utilisation	6
7.2	Diagramme de Classes	6

1. Présentation du projet

1.1 Présentation du contexte

Ce projet consiste à développer une solution logicielle pour optimiser l'organisation du travail en équipe. Il répond au besoin croissant des entreprises de structurer leurs processus de développement via des méthodes Agiles (Scrum/Kanban).

1.2 Nom du site

TASK-FLOW (Nom provisoire)

1.3 Cible adressée par le site

Cette application s'adresse aux équipes de développement, aux chefs de projets et aux startups cherchant une alternative légère et performante aux outils complexes du marché.

1.4 Cadre

Projet de fin d'année / Projet Fil Rouge.

1.5 Périmètre du projet

Le projet vise à fournir une application Web complète (SaaS). L'application sera entièrement *Responsive Design* via Tailwind CSS. Toutes les fonctionnalités de gestion de projet (Sprints, Kanban) seront accessibles sur mobile et desktop.

1.6 Description

L'application web est une plateforme de gestion de projets collaborative. Elle vise à reproduire un environnement de travail professionnel permettant de découper des projets en *Sprints* (cycles) et en *Cartes* (tâches). L'objectif principal est la clarté : visualiser qui fait quoi et quand. L'application permet de créer des espaces de travail, d'inviter des collaborateurs et d'assigner des rôles précis. Le cœur du système est le tableau interactif où les tâches évoluent de *À faire* à *Terminé*.

2. Description graphique et ergonomique

2.1 Charte graphique

L'interface sera développée avec Tailwind CSS pour garantir un design moderne, épuré et cohérent. L'accent sera mis sur la lisibilité (typographie claire) et l'utilisation de codes couleurs universels pour les statuts :

- **Gris** — Tâche en attente (*To Do*)
- **Bleu** — Tâche en cours (*Doing*)
- **Vert** — Tâche terminée (*Done*)

- **Rouge** — Priorité haute / Bloquant

2.2 Inspiration

- **Jira** — pour la structure des données
- **Linear** — pour le minimalisme et la rapidité
- **Trello** — pour la simplicité du Kanban

2.3 Logo

Le logo représentera la fluidité et l'interconnexion (ex : deux flux qui se rejoignent).

3. Objectifs du site

- Mettre à disposition une application web de gestion de projets (CRUD complet).
- Permettre la collaboration en temps réel sur l'avancement des tâches.
- Gérer une hiérarchie stricte d'utilisateurs (Admin, Manager, Membre).
- Offrir une expérience utilisateur fluide (Single Page Application).
- Le Back-office (API) doit sécuriser l'accès aux données selon les rôles.

4. Besoins Fonctionnels

L'application offre une suite d'outils pour gérer le cycle de vie complet d'un projet logiciel, de la conception à la livraison.

4.1 Arborescence du site

L'application est une SPA (Single Page Application), la navigation est instantanée :

- **Tableau de bord (Dashboard)** : Liste des projets accessibles.
- **Le Board (Kanban)** : Vue principale avec colonnes et cartes.
- **Backlog** : Liste des tâches en attente de planification.
- **Gestion d'équipe** : Administration des membres et rôles.
- **Page de Connexion / Inscription**.

4.2 Exposition des projets (Le Board)

Le cœur de l'application. Il s'agit d'un tableau virtuel divisé en colonnes (*To Do*, *Doing*, *Done*). Chaque tâche est représentée par une carte affichant le titre, la priorité, et l'avatar de la personne assignée. L'interface doit permettre une lecture rapide de l'état du projet.

4.3 Inscription et Gestion des Rôles (RBAC)

L'accès est sécurisé. Le système repose sur des rôles définis en base de données :

- **Administrateur** : Propriétaire du projet. Il configure l'espace, gère les sprints et a tous les droits sur les cartes.
- **Rôle Intermédiaire (Manager)** : Peut créer/modifier des sprints, assigner des tâches aux autres et valider le travail.
- **Rôle Standard (Membre)** : Visualise le projet. Il ne peut modifier et déplacer que les cartes qui lui sont assignées. Il peut créer des tickets de bugs.

4.4 Création d'une tâche

Lorsqu'un utilisateur crée une tâche, il remplit le formulaire manuellement en renseignant le titre, la description et le niveau de priorité de la tâche.

4.5 Attribution et Workflow

Une fois la tâche créée, elle est assignée à un utilisateur. Le changement de statut (déplacement d'une colonne à l'autre) déclenche une mise à jour en base de données. L'interface doit réagir immédiatement pour confirmer le déplacement.

4.6 Commentaires

Les utilisateurs peuvent laisser des commentaires techniques sur chaque carte pour centraliser la communication et éviter les échanges externes.

4.7 Sécurité

L'application manipule des données professionnelles. L'API Laravel doit utiliser Sanctum pour l'authentification par token. Chaque requête de modification (ex : supprimer une tâche) doit être validée par une *Policy* côté serveur pour vérifier si l'utilisateur possède le rôle requis.

5. Choix technologiques

5.1 Front-End

- **Framework** : React
- **Styling** : Tailwind CSS — pour un développement rapide et un design responsive natif.

5.2 Back-End

- **Framework** : Laravel
- **Base de données** : PostgreSQL

5.3 Compatibilité

L'application doit être compatible avec tous les navigateurs modernes : Google Chrome, Firefox, Safari, Edge et Opera.

6. Planification du projet

La gestion, le suivi de l'avancement et la distribution des tâches seront réalisés via l'outil Jira. Le cycle de développement adopte une approche itérative inspirée de la méthodologie Agile, découpant la réalisation en quatre *Sprints* majeurs.

6.1 Sprint 1 : Conception

- Définition de l'architecture logicielle globale du système.
- Modélisation de l'application via les diagrammes UML.
- Définition du schéma de la base de données relationnelle.
- Validation finale des choix techniques et du périmètre fonctionnel.

6.2 Sprint 2 : Maquettage (UI/UX)

- Création des wireframes et des maquettes haute fidélité pour les vues desktop et mobile.
- Définition de la palette de couleurs, de la typographie et des composants réutilisables.
- Intégration des premières interfaces statiques en utilisant Tailwind CSS.

6.3 Sprint 3 : Back-End

- Configuration de l'environnement de développement et initialisation du projet Laravel.
- Création des migrations, des modèles et de la base de données PostgreSQL.
- Développement de l'API RESTful et mise en place de l'authentification sécurisée avec Laravel Sanctum.
- Implémentation du système de permissions (RBAC) et de la logique métier (gestion des statuts, assignments).

6.4 Sprint 4 : Front-End

- Initialisation du projet React.
- Consommation de l'API Laravel et gestion de l'état global de l'application.
- Développement des fonctionnalités dynamiques : drag-and-drop sur le Kanban (Board), mise à jour en temps réel des statuts.
- Phase de tests, correction de bugs (débugage) et optimisation des performances.

7. Conception UML

Afin de garantir une architecture robuste et une vision claire du système avant d'entamer le développement, la modélisation logicielle sera réalisée avec l'outil Visual Paradigm. Deux diagrammes principaux viendront formaliser les besoins et la structure des données.

7.1 Diagramme des Cas d'Utilisation

Ce diagramme illustre les interactions directes entre les différents types d'utilisateurs (acteurs) et les fonctionnalités offertes par la plateforme.

- **Acteurs identifiés** : Administrateur, Manager, Membre.
- **Interactions principales** : Authentification, création d'un espace de travail, gestion des Sprints, manipulation des cartes (création, édition, déplacement), assignation des tâches.

7.2 Diagramme de Classes

Ce diagramme détaille la structure statique de l'application, modélisant les entités métier au cœur de la base de données et les associations qui les lient.

- **Entités clés** : Utilisateur, Rôle, Projet, Sprint, Tâche (Carte), et Commentaire.
- **Associations et multiplicités** : Un projet regroupe plusieurs sprints ; un sprint contient plusieurs tâches ; une tâche est assignée à un utilisateur précis et peut accumuler plusieurs commentaires.
- **Attributs spécifiques** : Utilisation d'énumérations pour définir de manière stricte les différents statuts possibles d'une tâche (*To Do*, *Doing*, *Done*) et son niveau de priorité.